

Paralel Programlama ile Point in Polygon Analizi

RAY-CASTING ALGORİTMASI & ÇOKLU THREAD OPTİMİZASYONU

TALHA KORKMAZ

4. SINIF | 23360859739
BİLGİSAYAR MÜHENDİSLİĞİ

Algoritma ve Mimari

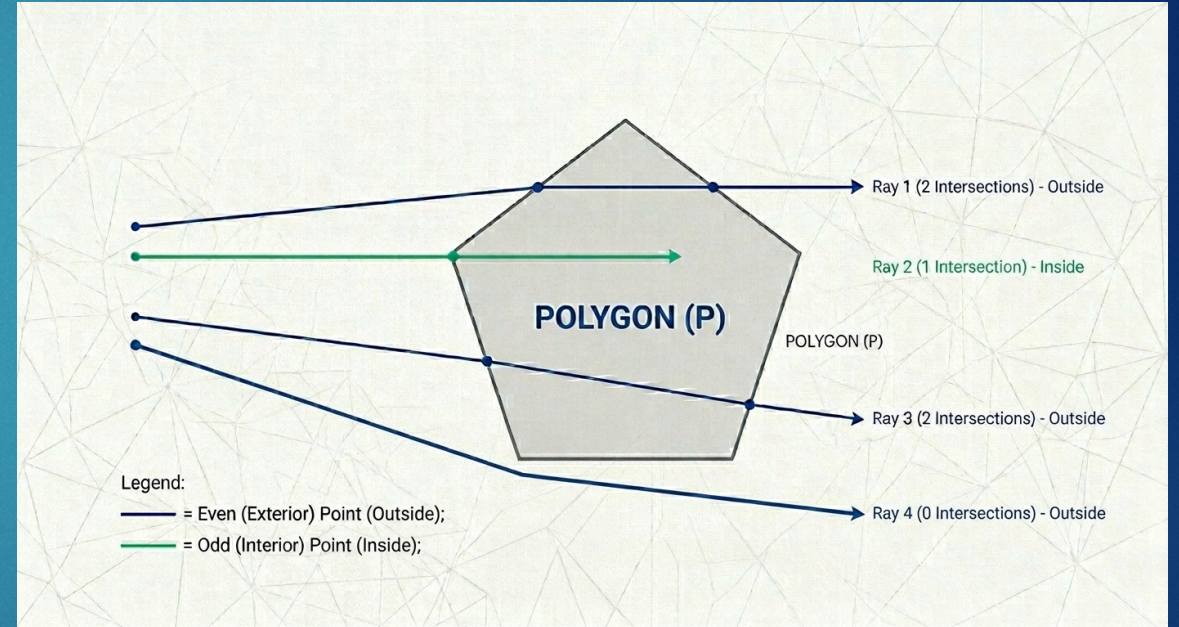
- Milyonlarca noktanın geometrik konum tespiti



Ray-Casting Algoritması

Noktanın poligon içinde olup olmadığını anlamak için kullanılan en verimli yöntemdir.

- • **Mantık:** Noktadan sağa doğru sanal bir ışın gönderilir.
- • **Kesişim:** Poligon kenarlarıyla kesişim sayısı hesaplanır.
- • **Karar:** Tek sayıda kesişim "İçeride", çift sayıda kesişim "Dışarıda" demektir.



Java Uygulama Bileşenleri

Point Sınıfı

X, Y koordinatlarını ve hesaplanan boolean durumunu tutan temel veri yapısı.

WorkerThread

Thread sınıfından türetilen ve veri bloklarını paralel işleyen çekirdek yapı.

Clone Logic

Testlerin birbirinden etkilenmemesi için bellek seviyesinde veri izolasyonu.

Paralel Çözüm Stratejisi

Veri Bloklama (Chunking)

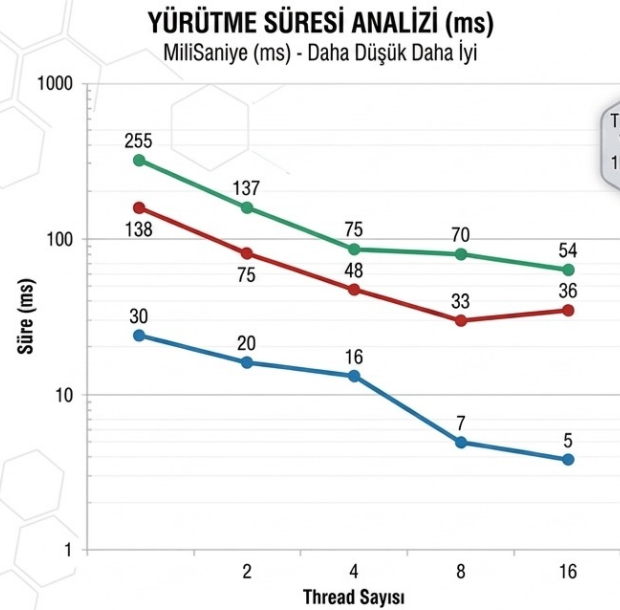
- 5 Milyon nokta, aktif thread sayısına bölünerek her çekirdeğe eşit yük dağıtılır. Bu sayede "Embarrassingly Parallel" yapısından maksimum fayda sağlanır.

Thread Senkronizasyonu

- `threads[i].join()` komutu ile ana thread'in tüm işçileri beklemesi sağlanır. Yarış durumu (Race Condition) oluşmadan veri bütünlüğü korunur.

Performans Analizi

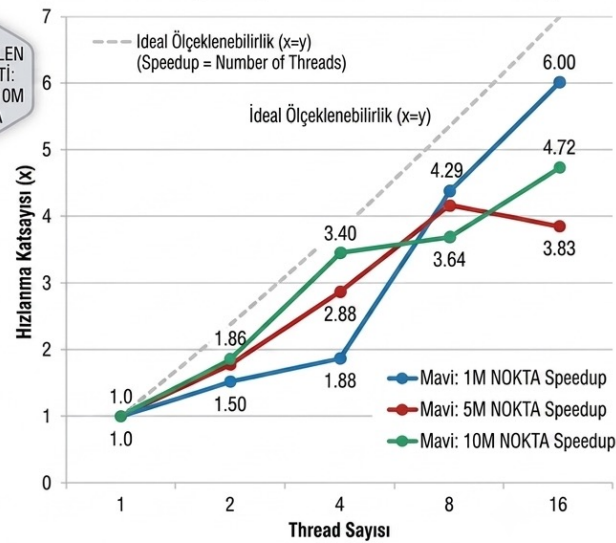
- ▶ 1M, 5M ve 10M nokta setleri üzerinde ampirik sonuçlar



TEST EDİLEN
VERİ SETİ:
1M, 5M, 10M
NOKTA

HIZLANMA KATSAYISI (SPEEDUP) VE ÖLÇKELENEBİLİRLİK

Hızlanma = Ardışıl Süre / Paralel Süre - Daha Yüksek Daha İyi



Kapsamlı Performans Verileri



PARALEL PROGRAMLAMA
PROJE ANALİZİ



JAVA THREAD
KULLANIMI



JAVA POLİGONUN
HIZLANMA KATSAYISI



NOKTA POLİGONUN
İÇİNDE Mİ TESPİTİ



Maksimum Performans

6.00x

Zirve Hızlanma Oranı

Ölçeklenebilir Verimlilik

Veri boyutu arttıkça, thread yönetim maliyetinin toplam süreye oranı azalmakta ve paralel işlem verimliliği %20 oranında daha iyi sonuç vermektedir.

Donanım Sınırları

Context Switching & Overhead

- ▶ 16 Thread kullanımında 5M nokta setinde gözlenen süre artışı, fiziksel çekirdek sınırına ulaşıldığını kanıtlar.
- ▶ İşletim sisteminin thread'ler arası geçiş maliyeti (Overhead), hesaplama kazancının önüne geçtiğinde doygunluk noktası oluşur.

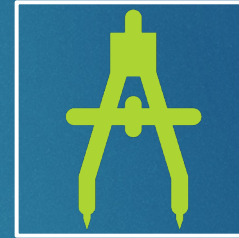
Sonuç ve Doğrulama



%100 Doğruluk: Paralel ve ardışıl sonuçlar tüm testlerde birebir örtüşmüştür.



Veri Paralelliği: Algoritma, yüksek veri setlerinde mükemmel ölçeklenmiştir.



Hyper-Threading: Mantıksal çekirdeklerin 10M noktada performansı %15 artırdığı gözlemlenmiştir.

Sorularınız?

Dinlediğiniz için teşekkürler.